FIG. 16A — PROVING POOL — ARCHITECTURE, IPC PROTOCOL & JOB DISPATCH 1600

SECTION A: POOL ARCHITECTURE OVERVIEW 1600

MAIN PROCESS (COORDINATOR):

```
ProvingPool (poolSize:N, jobQueue:FIFO,
   workers:Map, pendingJobs:Map)
L2SequencerService
   prepareBatch() / finalizeBatch() / restoreDrained()
L2Node
   executeBatchWithPool() / setProvingPool()
```

State held in main ONLY:
Merkle trees, receipts,
batches, audit chain,
claim accumulator,
message store

IPC (fork) IPC (fork) IPC (fork)

Worker 0	Worker 1	Worker 2
o1js context	o1js context	o1js context
V8 Isolate	V8 Isolate	V8 Isolate
WASM heap (own)	WASM heap (own)	WASM heap (own)
StateTransitionProg	StateTransitionProg	StateTransitionProg
BatchProgram	BatchProgram	BatchProgram
ChainProgram	ChainProgram	ChainProgram

Workers are STATELESS — only compiled ZkPrograms. TRUE CPU parallelism via child_process.fork()

SECTION B: IPC MESSAGE PROTOCOL 1602

COORDINATOR -> WORKER:

Dir	Type	Payload
C->W	compile	(none)
C->W	prove	jobId, ProvingJobRequest
C->W	shutdown	(none)

WORKER -> COORDINATOR:

Dir	Type	Payload
W->C	ready	compileTimeMs
W->C	result	jobId, ProvingJobResult
W->C	error	jobId, message

IPC SERIALIZATION BOUNDARY: All Field values -> decimal strings. All proofs -> proof.toJSON().
All BigInts -> .toString(). NO o1js objects cross the IPC boundary — only JSON-safe data.

SECTION C: JOB DISPATCH AND FIFO QUEUE 1604

```
pool.prove(request) -> Promise<ProvingJobResult>
   |
   Idle worker available?
   |
   YES                                   NO
   |                                     |
   dispatchJob(worker, job):             jobQueue.push(job) (FIFO)
      worker.busy = true                 When ANY worker finishes:
      pendingJobs.set(id)                  drainQueue() -> pop next
      Set proveTimeout (5 min)             -> dispatch to idle worker
      Send ProveMessage via IPC
         |
         SUCCESS: ResultMessage -> resolve promise -> recycle check
         ERROR: ErrorMessage -> reject promise -> drainQueue()
         TIMEOUT: 5 min timer -> reject promise -> drainQueue()
```

CRASH HANDLING:
Worker exits unexpectedly
1. Reject in-flight job
2. totalJobsFailed
3. restartCount
4. restartCount < 3?
   YES: spawn -> compile -> drainQueue
   NO: worker permanently removed

TIMEOUT HANDLING:
proveTimeout fires (5 min)
1. Clear from pendingJobs
2. worker.busy = false
3. totalJobsFailed
4. job.reject(Error)
5. drainQueue()

All workers compile same programs — any worker can prove any job. No affinity.